Operációs rendszerek

5. Ütemezés

v5.0

Tartalom

Bevezetés
CPU ütemezés
Ütemezési algoritmusok alapjai
Ütemezési algoritmusok

5.1 Bevezetés

OS legfontosabb feladata

- a rendszer erőforrásainak kezelése
  - hatékony gazdálkodás
  - védelem
- erőforrás biztosítása a folyamatoknak
  - kényelem
  - biztonság
  - fairness

Ütemezés

Definíció:

Az ütemezés (scheduling) az a tevékenység, amelynek eredményeként eldől, hogy az adott erőforrást a következő pillanatban mely folyamat használhatja.

5.2 CPU ütemezés

Három különböző szint van:

- hosszútávú (long term)
- középtávú (medium term)
- rövidtávú (short term)

Nem minden OS-ben van meg mindegyik típusú ütemezés.

Hosszú távú ütemezés

A háttértáron várakozó, még el nem indított munkák közül melyek kezdjenek futni.

- Ritkán fut, így nem kell, hogy gyors legyen.
- Követelmények:
  - Olyan munka-halmaz (*job-mix*) előállítása, amely a rendszer erőforrásait kiegyensúlyozottan használja.
  - CPU-korlátozott és periféria-korlátozott munkák egyenletesen forduljanak elő.

Középtávú ütemezés

A rendszer időszakos terhelés-ingadozásait próbálja kiegyenlíteni.

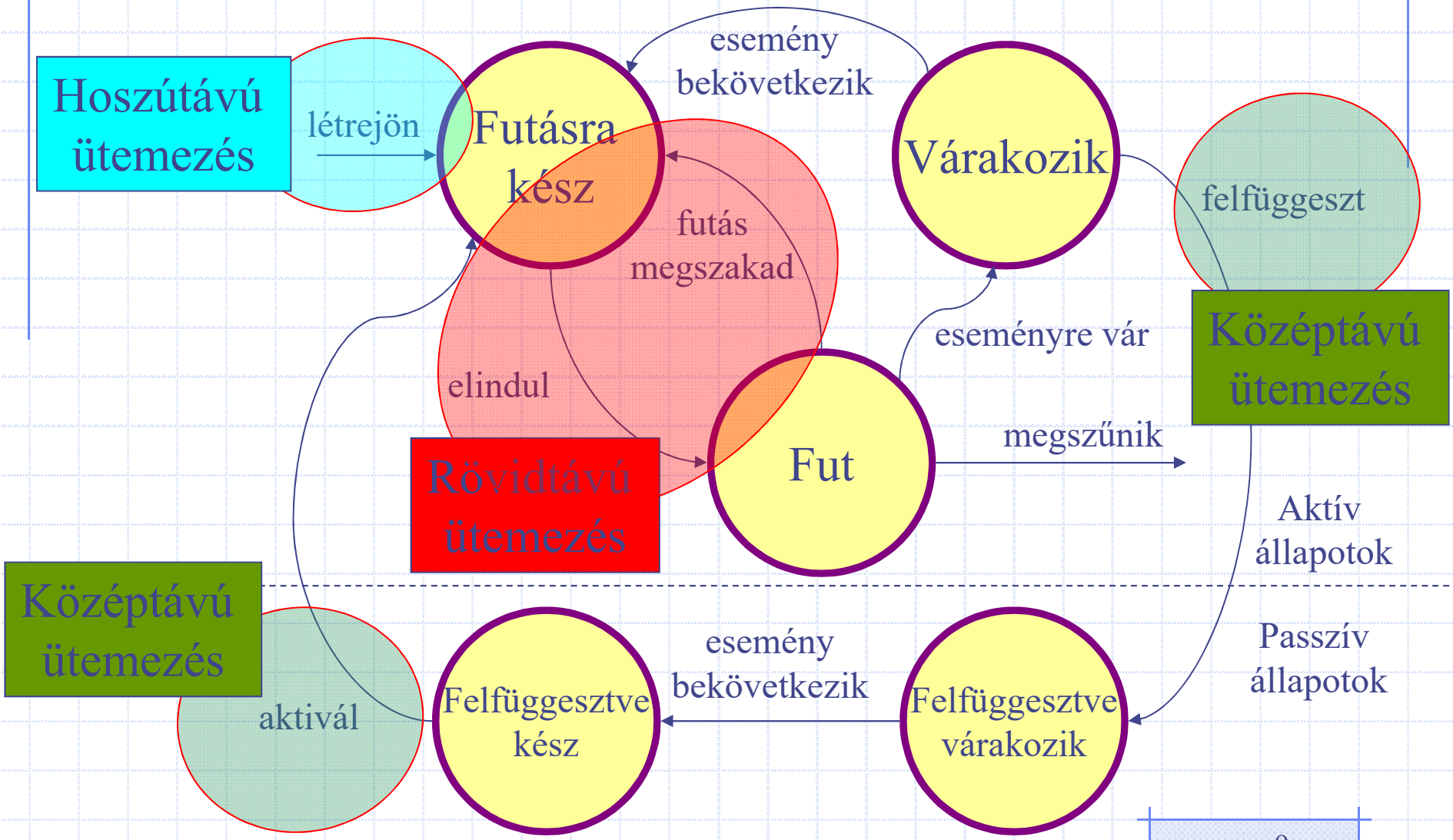
- Folyamatok felfüggesztése/aktiválása
  - Felfüggesztés esetén folyamat a háttértáron tárolódik, megfosztják elvehető erőforrásaitól.
  - Az ilyen folyamatok nem versengenek tovább az erőforrásokért.

Rövidtávú ütemezés

Melyik futásra kész folyamat kapja meg a CPU-t

- Gyakran fut, ezért gyorsnak kell lennie, különben a rendszer túl sok időt töltene az ütemezéssel, elvéve a CPU-t a folyamatoktól.
- Az ütemező mindig a tárban van, része a OS magjának.

Emlékeztető: folyamatok kibővített állapotai



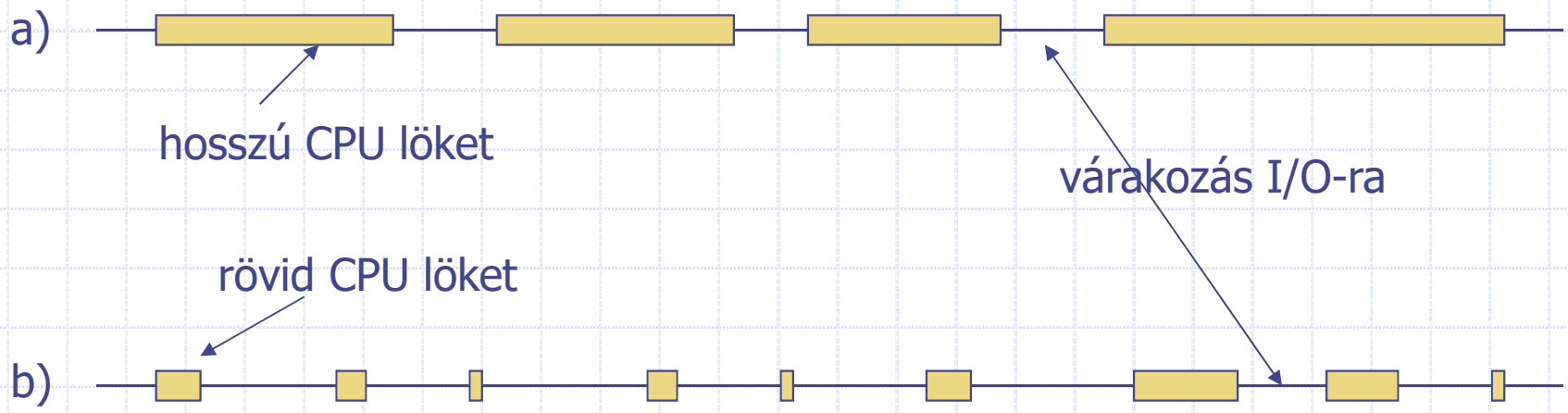
5.3 Az ütemezési algoritmusok alapjai

- Egy folyamat futása során két különböző jellegű tevékenységet végez:
  - CPU löket: a folyamatnak csak a CPU és a központi tár kell
  - I/O löket: a folyamat perifériás átvitelt hajt végre, CPU nem kell neki
- CPU, I/O löket átlagos hossza folyamatonként más és más

A CPU löket és az IO löket

CPU és I/O löketek váltakoznak

a) CPU-korlátos folyamat
b) I/O-korlátos folyamat



A CPU löket és az IO löket 2.

load store
add store
read from file

wait for I/O

store increment
index
write to file

wait for I/O

load store
add store
read from file

wait for I/O

CPU löket

I/O löket

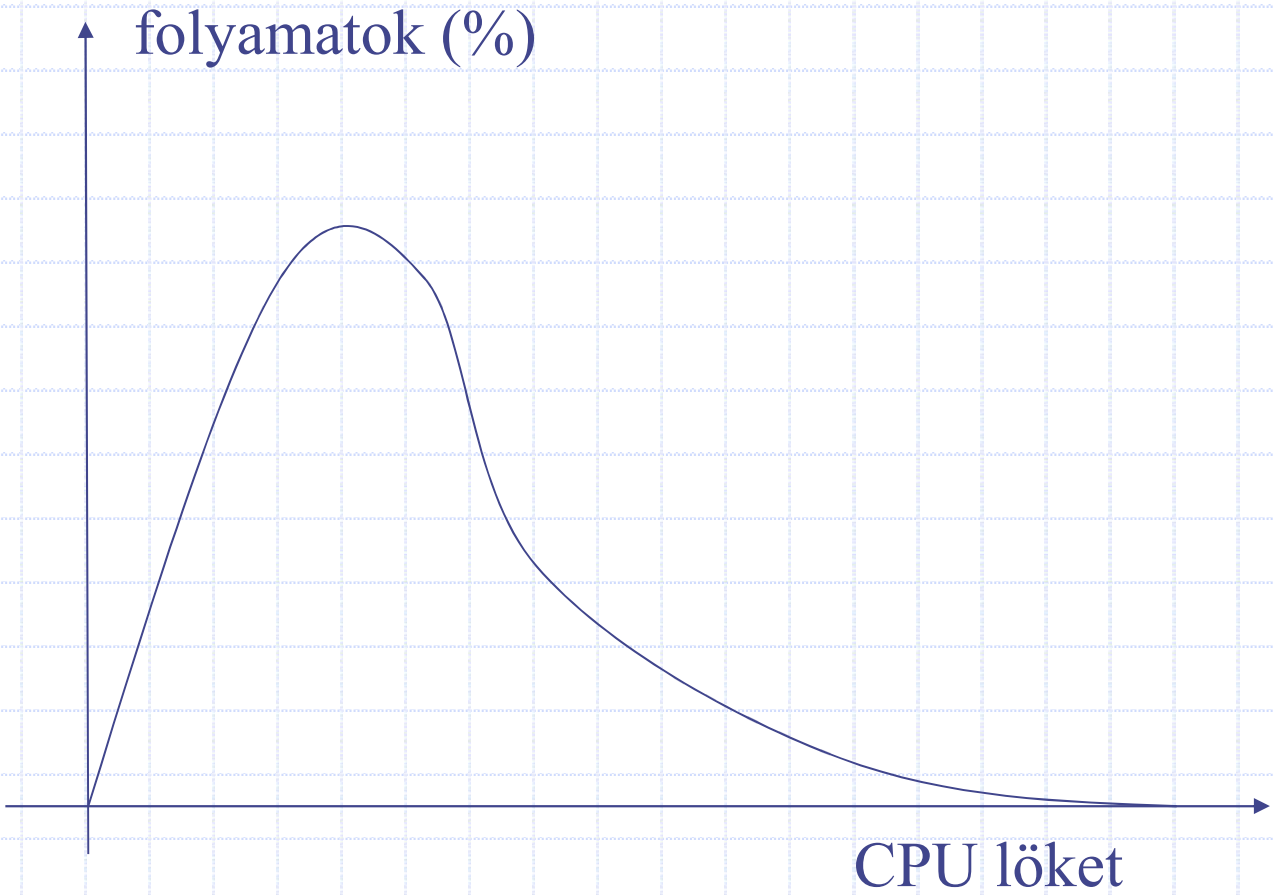
CPU löket

I/O löket

CPU löket

I/O löket

A CPU löket eloszlása

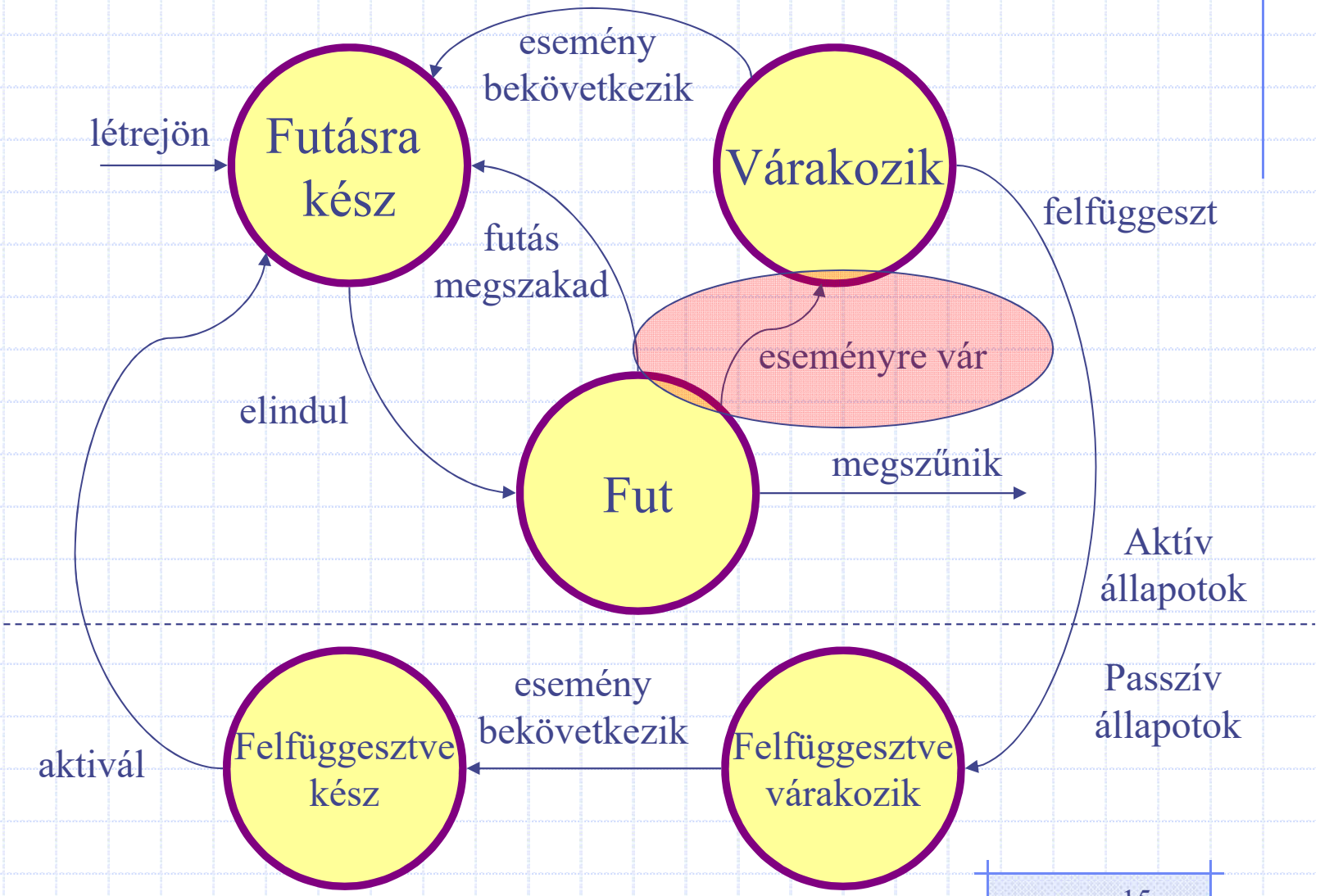


Mikor történik ütemezés?

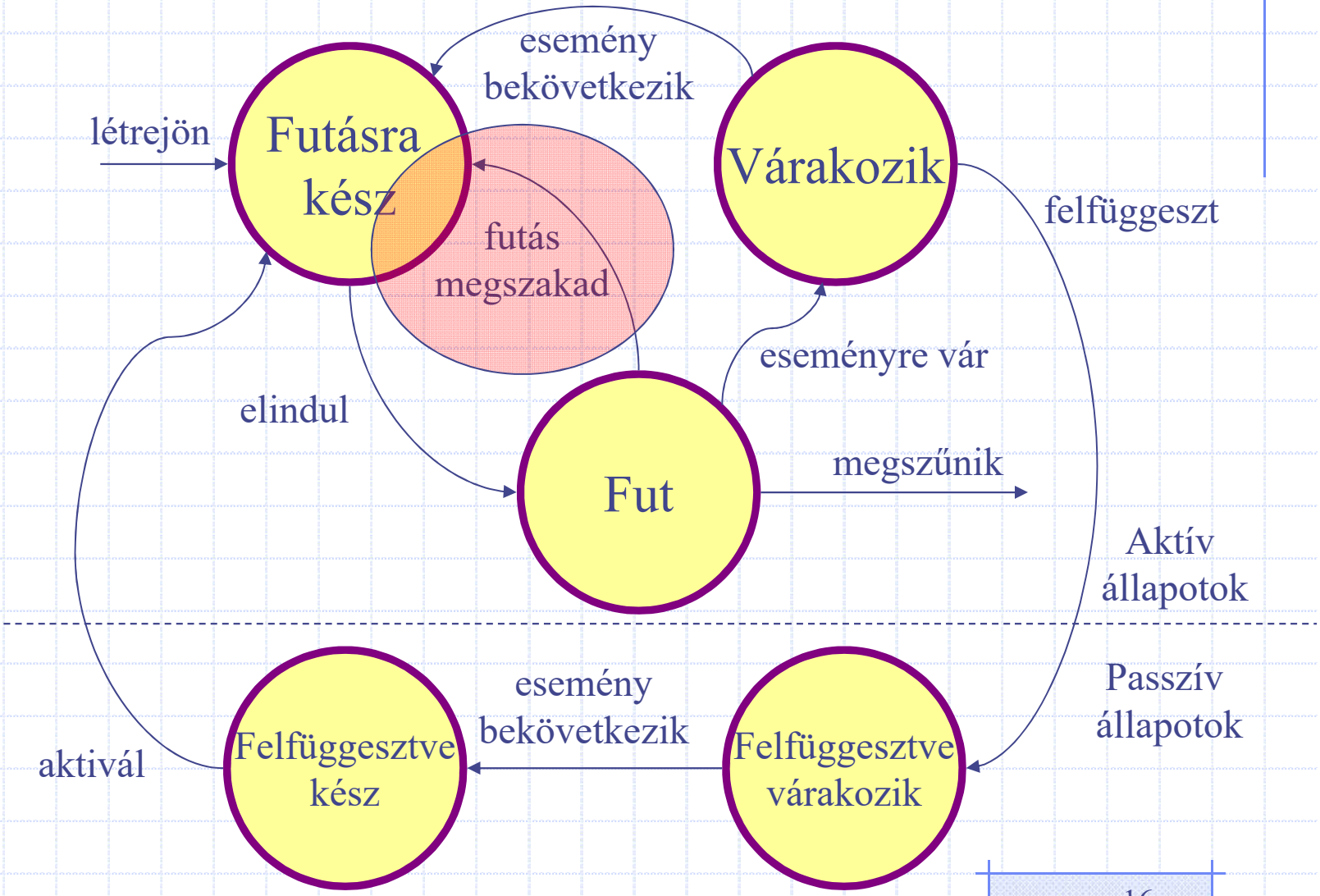
Ütemezés a következő állapotátmeneteknél következik be:

- a futó folyamat várakozni kényszerül
- a futó folyamat lemond a CPU-ról vagy elveszik tőle
- a folyamat futásra késszé válik (nem mindig van ütemezés)
- a futó folyamat befejeződik

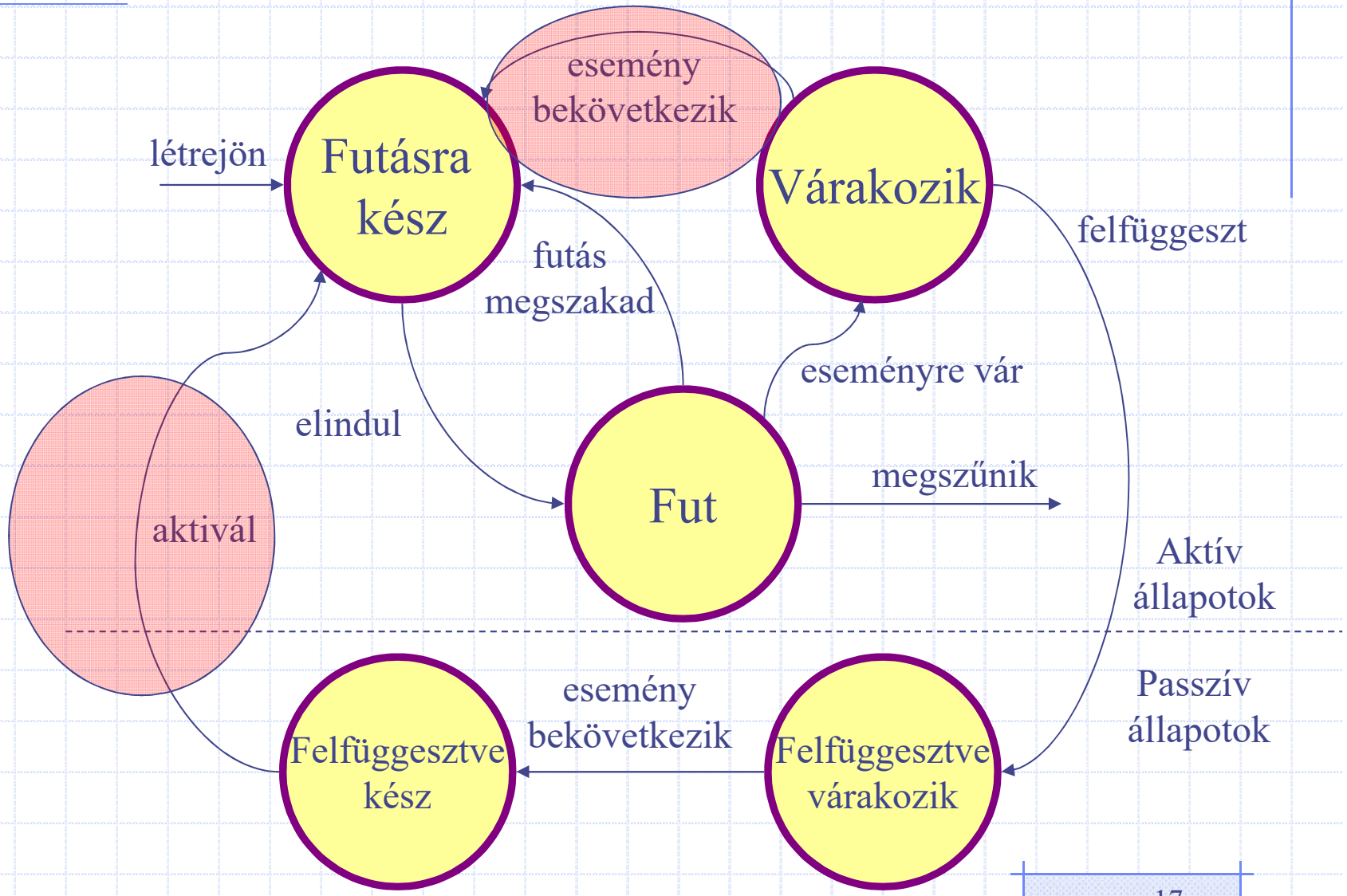
1. A futó folyamat várakozni kényszerül



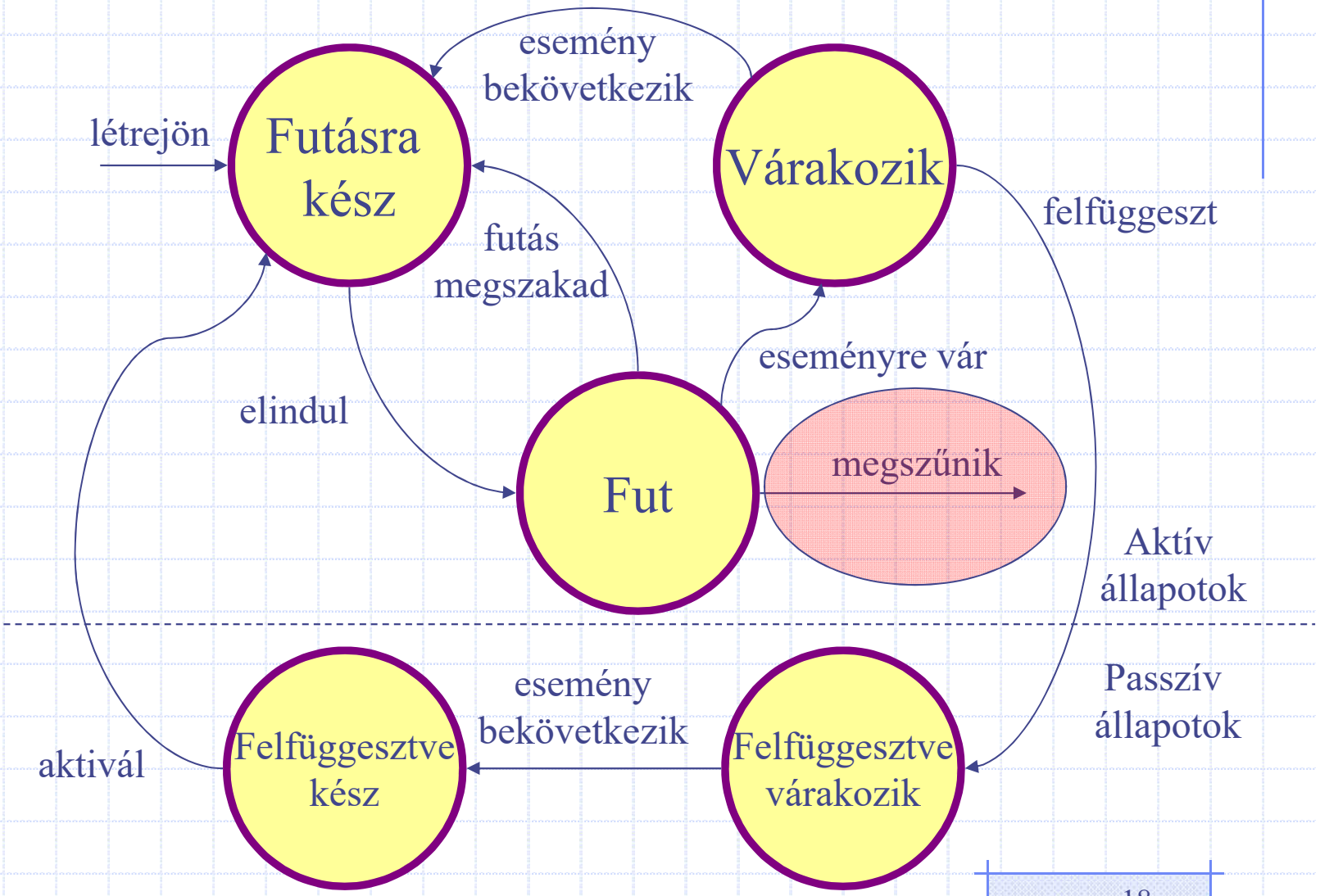
2. A futó folyamat lemond a CPU-ról, vagy elveszik tőle



3. A folyamat futásra késszé válik



4. A futó folyamat befejeződik



Ütemezés és környezetváltás

Ütemezés

1. a futó folyamat várakozni kényszerül
2. futó folyamat lemond a CPU-ról vagy elveszik tőle
3. folyamat futásra késszé válik
4. a futó folyamat befejeződik

Környezetváltás

- Az 1. és 4. esetben *mindig* van környezetváltás
- A 2. és 3. esetben *nem mindig* van környezetváltás.

Ütemezés típusai

Nem preemptív (kooperatív): a folyamattól nem lehet elvenni a CPU-t

- a folyamat csak az általa kiadott utasítások hatására vált állapotot
  - erőforrásra, eseményre várakozás
  - befejeződés vagy
  - a CPU-ról önként lemondás

Preemptív: az OS elveheti a futás jogát a folyamattól

- futásra kész állapotba teszi a futó folyamatot
- egy másik (futásra kész) folyamatot indít el

Az ütemezési algoritmusok teljesítményének mérése I.

CPU kihasználtság (CPU utilization)

A CPU az idejének hány százalékát használja a folyamatok utasításainak végrehajtására.

Kihasználtságot csökkenti: CPU henyél (idle), rendszer-adminisztrációra fordított idő (rezsi) sok.

Átbocsátó képesség (throughput)

Az OS időegységenként hány munkát futtat le.

Körülfordulási idő (turnaround time)

Egy munka a rendszerbe helyezéstől számítva mennyi idő alatt fejeződik be.

Az ütemezési algoritmusok teljesítményének mérése II.

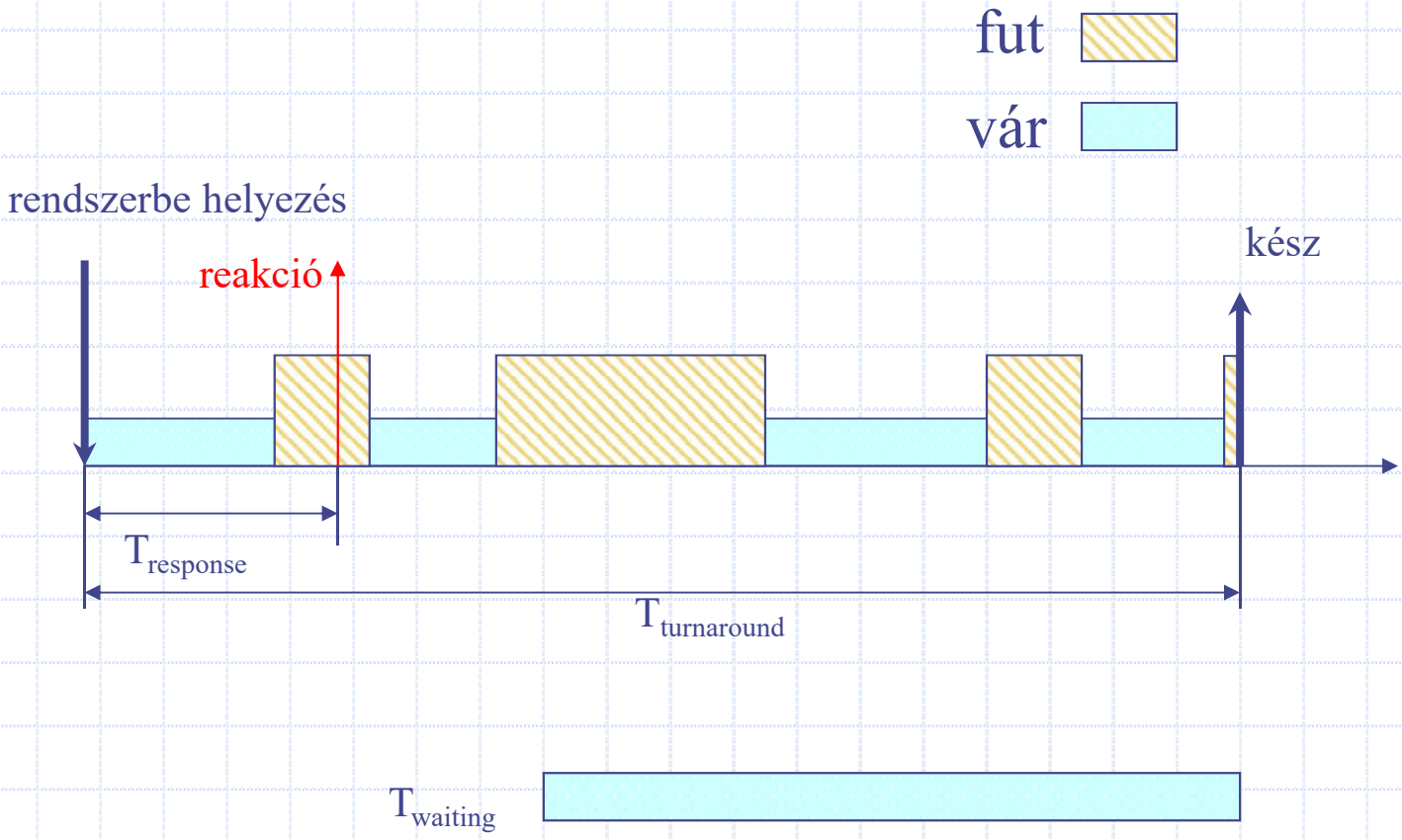
Várakozási idő (waiting time)

Egy munka (vagy folyamat) mennyi időt tölt várakozással (futásra kész állapot, várakozó állapot, felfüggesztett állapotok, (long-term) előzetes várakozás).

Válaszidő (response time)

Időosztásos (interaktív) rendszereknél fontos. Az OS reakció ideje, a kezelői parancs után a rendszer első látható reakciójáig eltelt idő.

Teljesítménymérés



Az ütemezési algoritmusokkal szembeni követelmények 1.

- Valamely (előbbi paraméterekből képzett) célfüggvény szerint legyen **optimális**
- Legyen korrekt, kezeljen minden folyamatot (vagy bizonyos típusú folyamatokat) azonos módon (**igazságos**).
- Biztosítson egyes folyamatoknál **prioritás**t
- Kerülje el a folyamatok **kiéheztetés**ét
- Legyen **megjósolható** viselkedésű (meg lehessen becsülni a várható maximális körülfordulási időt)
- Minimalizálja a **rezsi idő**t (gyakran kis többlet adminisztrációval jobb általános rendszerteljesítmény érhető el)

Az ütemezési algoritmusokkal szembeni követelmények 2.

- **részesítse előnyben**
  - a kihasználatlan erőforrásokat igénylő folyamatokat
  - a fontos erőforrásokat foglaló folyamatokat
- növekvő terhelés esetén a rendszer teljesítménye "elegánsan", **fokozatosan romoljon le** (graceful degradation), ne omoljon hirtelen össze.

- Sok cél egymásnak ellentmond, így azok együttesen nem teljesíthetők.
- Fontos megfogalmazni egy rendszer tervezése folyamán azokat a célokat, amelyek kiemelt fontosságúak.

5.4 Ütemezési algoritmusok

Egyszerű algoritmusok
Prioritásos algoritmusok
Többszintű algoritmusok
Többprocesszoros ütemezés

5.4.1 Egyszerű ütemezési algoritmusok

Legrégebben várakozó (First Come First Served, FCFS)

Legrövidebb (löket)idejű (Shortest Job First, SJF)

Körforgó (Round-Robin, RR)

Legrégebben várakozó

First Come First Served (FCFS)

- A futásra kész folyamatok a várakozási sor végére kerülnek, az ütemező a sor elején álló folyamatot kezdi futtatni.
- Nem preemptív.
- Egyszerűen megvalósítható
- Konvoj hatás (egy hosszú CPU löketű folyamat feltartja a mögötte várakozókat).

Példa: FCFS

P_i	C_i (ms)
P_1	24
P_2	3
P_3	3

Végrehajtási sorrend: P_1, P_2, P_3

Átlagos várakozási idő:

Átlagos körülfordulási idő:

Átbocsátó képesség:

Megoldás: FCFS

P_i	C_i (ms)
P_1	24
P_2	3
P_3	3

Konvoj hatás!

Végrehajtási sorrend: P_1, P_2, P_3

- Átlagos várakozási idő: (24+27)/3=17ms
- Átlagos körülfordulási idő: (24+27+30)/3=27ms
- Átbocsátó képesség: 3job/30ms=1/10 job/ms

Konvoj hatás

Definíció:

A konvoj-hatás az a jelenség, amikor kooperatív ütemezés esetén egy nagy CPU-löketidejű futó folyamat mögött feltorlódnak a futásra kész folyamatok.

Kiküszöbölés:

- rövid CPU-löketidejű folyamatok először (SJF)
- időosztásos rendszer (RR)

Legrövidebb (löket)idejű

Shortest Job First (SJF)

- Nem preemptív algoritmus, a futásra kész folyamatok közül a legrövidebb löketidejűt indítja.
- Nincs konvoj hatás, optimális körülfordulási idő, optimális várakozási idő.
- Alkalmazása:
  - Hosszú távú ütemezés
  - Rövid távú ütemezés (RT rendszerek)

Példa: SJF

P_i	C_i (ms)
P_1	24
P_2	3
P_3	3

Végrehajtási sorrend: P_2, P_3 , P_1

Átlagos várakozási idő:

Átlagos körülfordulási idő:

Átbocsátó képesség:

Megoldás: SJF

P_i	C_i (ms)
P_1	24
P_2	3
P_3	3

Végrehajtási sorrend: P_2, P_3 , P_1

- Átlagos várakozási idő: (3+6)/3=3ms
- Átlagos körülfordulási idő: (3+6+30)/3=13ms
- Átbocsátó képesség: 3job/30ms=1/10 job/ms

Körforgó ütemezés

Round-Robin (RR)

- Preemptív algoritmus, az időosztásos rendszerek valamennyi ütemezési algoritmusainak az alapja.
- Folyamatok időszeletet kapnak (time slice).
  - Ha a CPU löket nagyobb mint az időszelet, akkor az időszelet végén az ütemező elveszi a CPU-t, a folyamat futásra kész lesz és beáll a várakozó sor végére.
  - Ha a CPU löket rövidebb, akkor a löket végén a folyamatokat újraütemezzük.

Körforgó ütemezés időszelete

- Időszelet meghatározása nehéz.
  - Nagy időszelet: FCFS algoritmushoz hasonló lesz.
  - Kis időszelet: folyamatok a CPU-t egyenlő mértékben használják, viszont a sok környezetváltás a teljesítményt rontja.
- **Ökölszabály**: a CPU löketek kb. 80%-a legyen rövidebb az időszeletnél.
- Általában 10-100ms

Példa: Round Robin

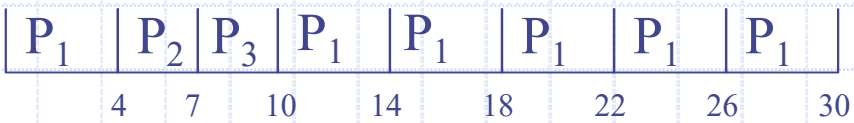
P_i	C_i (ms)
P_1	24
P_2	3
P_3	3

T_{slice}=4ms

Megoldás: Round Robin

P_i	C_i (ms)
P_1	24
P_2	3
P_3	3

T_{slice}=4ms



Átlagos várakozási idő: (6+4+7)/3=5,66ms

Átlagos körülfordulási idő: (30+7+10)/3=15,66ms

Átbocsátó képesség: 3job/30ms=1/10 job/ms

Összehasonlítás

FCFS

- Átlagos várakozási idő: (24+27)/3=17ms
- Átlagos körülfordulási idő: (24+27+30)/3=27ms
- Átbocsátó képesség: 3job/30ms=1/10 job/ms

SJF

- Átlagos várakozási idő: (3+6)/3=3ms
- Átlagos körülfordulási idő: (3+6+30)/3=13ms
- Átbocsátó képesség: 3job/30ms=1/10 job/ms

RR

- Átlagos várakozási idő: (6+4+7)/3=5,66ms
- Átlagos körülfordulási idő: (30+7+10)/3=15,66ms
- Átbocsátó képesség: 3job/30ms=1/10 job/ms

Round Robin időszelet hatása

- Az időszelet (T_{slice}) hatása a környezetváltás gyakoriságára
- C_i = 10ms

T_{slice}	Környezetváltások száma
12	0
6	1
1	9

5.4.2 Prioritásos ütemező algoritmusok

- A futásra kész folyamatokhoz egy prioritást (rendszerint egy egész számot) rendelünk.
- A legnagyobb prioritású folyamat lesz a következő futtatandó folyamat.

Prioritás

- Meghatározása
  - belső
    - az OS határozza meg
  - külső
    - az OS-en kívüli tényező (operátor, a folyamat saját kérése, stb.) határozza meg
- Futás során
  - statikus (végig azonos)
  - dinamikus (az OS változtathatja)

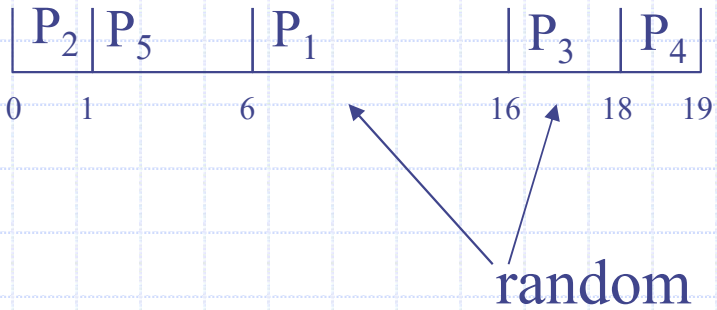
Példa: Statikus prioritás

Folyamat	C_i (ms)	Prioritás
P_1	10	3
P_2	1	1
P_3	2	3
P_4	1	4
P_5	5	2

Átlagos várakozási idő: ?

Megoldás: Statikus prioritás

Folyamat	C_i (ms)	Prioritás
P_1	10	3
P_2	1	1
P_3	2	3
P_4	1	4
P_5	5	2



Átlagos várakozási idő: (6+0+16+18+1)/5=8.2

A prioritás meghatározása

- Prioritást sokszor a löketidő alapján határozzák meg.
- A löketidő szükséglet meghatározása:
  - a folyamat (felhasználó) bevallása alapján (a „hazugságot” az OS később bünteti)
  - előző viselkedés alapján (a korábbi löketidők alapján becslés)

A kiéheztetés és elkerülése

- Kiéheztetés:
  - A folyamat sokáig (esetleg soha) nem jut processzorhoz
- Prioritásos algoritmusoknál kiéheztetés felléphet
- Ennek kivédése a folyamatok öregítése (*aging*):
  - a régóta várakozó folyamatok prioritását növeljük

Egy éhező folyamat

- Klasszikus példa: MIT, 1973
- Folyamat: 1967-ből

IBM 7094

Dinamikus prioritásos algoritmusok

- Legrövidebb hátralévő idejű
  - *Shortest Remaining Time First*, SRTF
- Legjobb válaszarány
  - *Highest Response Ratio*

Legrövidebb hátralévő idejű

Shortest Remaining Time First, SRTF

- Az SJF *preemptív* változata
- Ha egy új folyamat válik futásra késszé
  - akkor az ütemező újra megvizsgálja a futásra kész folyamatok, illetve az éppen futó folyamatot hátralévő löketidejét
  - és a legrövidebbet indítja tovább.
- A folyamat megszakítása és egy másik elindítása környezetváltozást igényel, ezt az időt is figyelembe kell vennünk.

Példa: SRTF

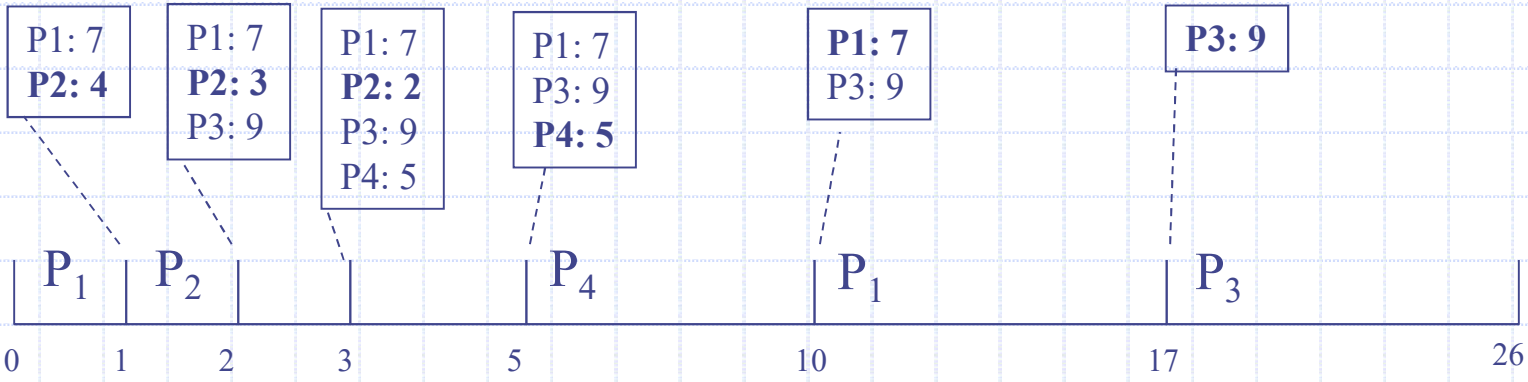
P_i	T_i (ms)	C_i (ms)
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Ütemezés?

Átlagos várakozási idő: ?

Példa megoldás: SRTF

P_i	T_i (ms)	C_i (ms)
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5



Átlagos várakozási idő: (9+0+15+2)/4=6.5ms

Legjobb válaszarány

Highest Response Ratio

- Az SJF algoritmus változata, ahol a várakozó folyamatok öregednek. A kiválasztás (a löketidő helyett) a

$$\frac{löketidő + k \cdot várakozási_idő}{löketidő}$$

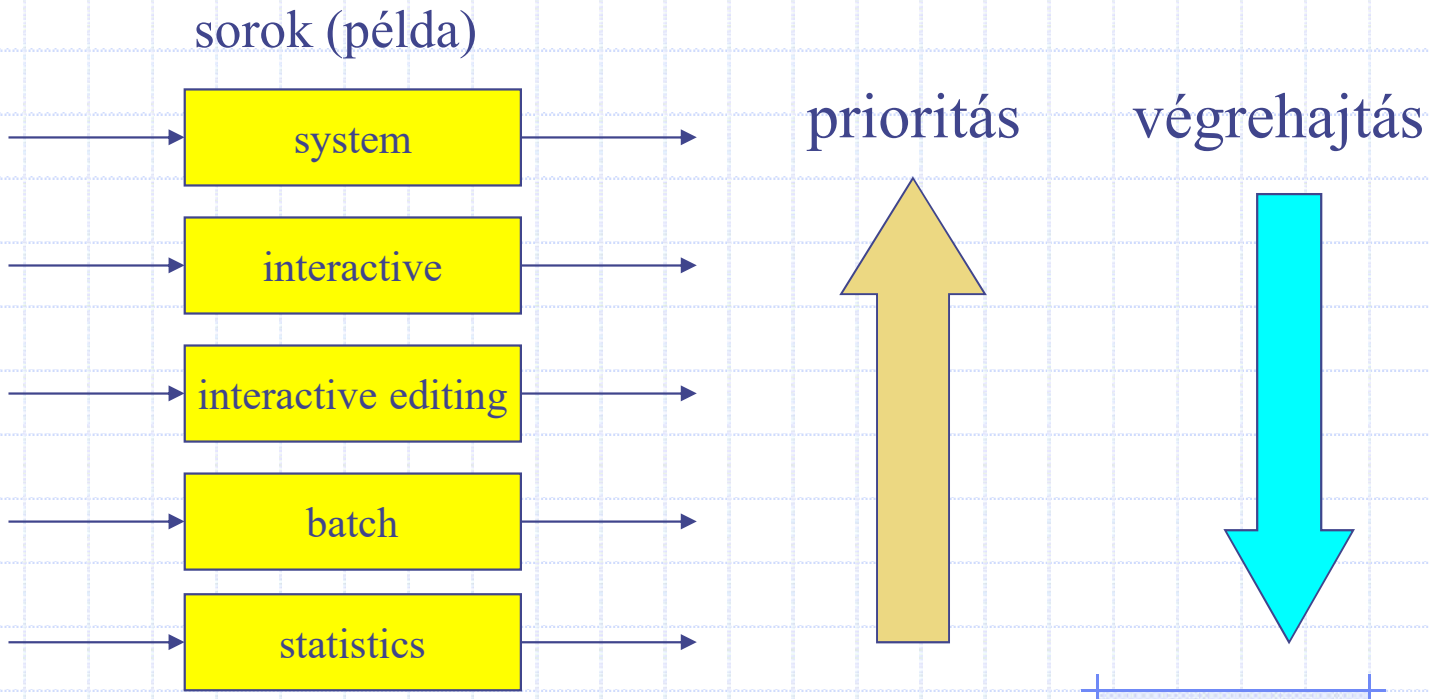
képlet szerint megy végbe, ahol *k* egy jól megválasztott konstans.

5.4.3 Többszintű algoritmusok

- Futásra kész folyamatokat több külön sorban várakoztatják.
- A sorokhoz prioritás van rendelve.
- Egy kisebb prioritású sorból csak akkor indulhat el egy folyamat, ha a nagyobb prioritású sorok üresek.
- Az egyes sorokon belül különböző kiválasztási algoritmusok működhetnek.
- Példák:
  - Statikus többszintű sorok
  - Visszacsatolt többszintű sorok

Statikus többszintű sorok

- *Static Multilevel Queue*, SMQ
- A folyamatot elindulásakor valamilyen kritérium alapján besorolunk egy várakozó sorba.
- A folyamat élete során végig ugyanabban a sorban marad.

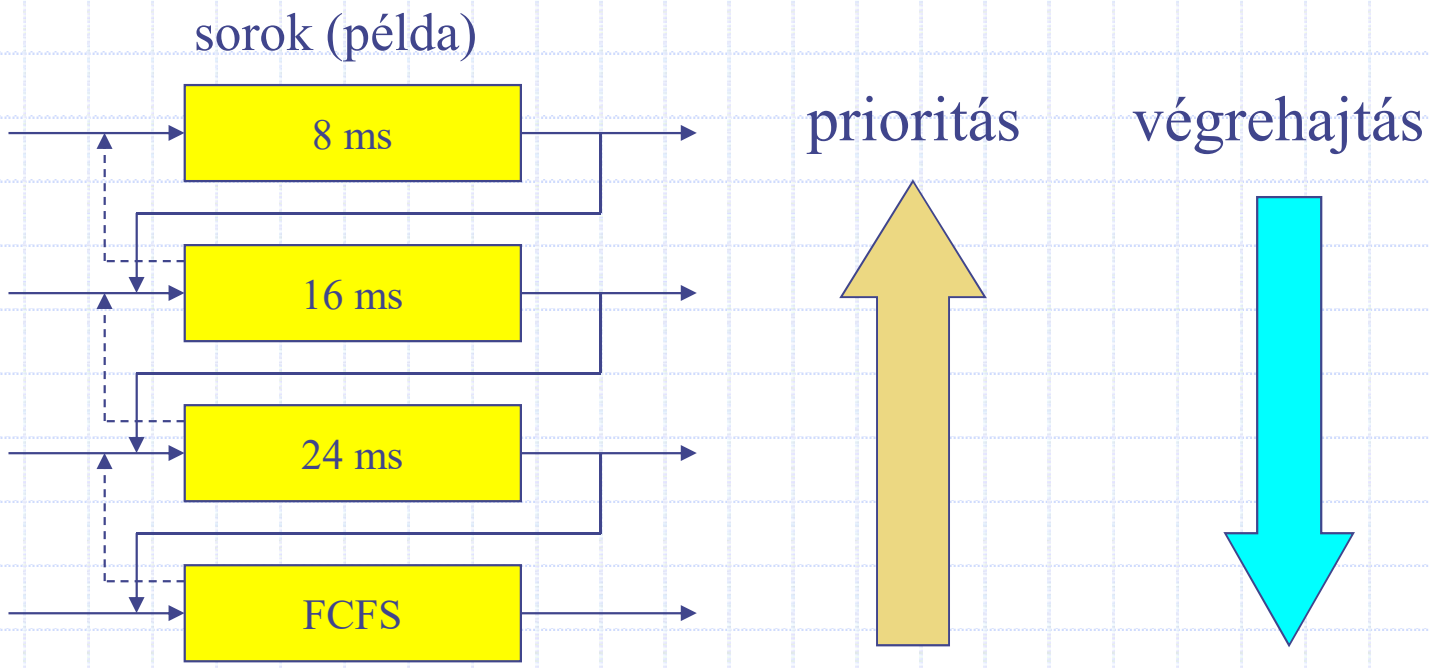


Statikus többszintű sorok

- Egy lehetséges példa a prioritások besorolására:
  - rendszer folyamatok (magas prioritás, közvetlen hatással vannak a rendszer működésére)
  - interaktív folyamatok (biztosítani kell a felhasználó számára az elfogadható válaszidőt)
  - interaktív szövegszerkesztők (kevésbé kritikusak)
  - kötegelt feldolgozás (általában akkor futnak, ha "van idő")
  - rendszerstatisztikákat gyűjtő folyamatok (alacsony prioritás, nincsenek közvetlen hatással a rendszer működésére)

Visszacsatolt többszintű sorok

- *Multilevel Feedback Queues*, MFQ
- A sorokhoz egy időszelet tartozik:
  - minél nagyobb a prioritás, annál kisebb az időszelet
- A folyamatok futásuk során átkerülhetnek másik sorokba



Visszacsatolt többszintű sorok

A folyamatok sor- és prioritás-váltása

- Nagyobb prioritásúból kisebb prioritású sorba:
  - Amennyiben nem elég az adott időszelet, a folyamat egy kisebb prioritású sorba kerül át.
- Kisebb prioritásúból nagyobb prioritású sorba:
  - A folyamat átlagos löketidejének változása (csökkenése) esetén
  - A régóta várakozó folyamat öregedése miatt

5.5 Többprocesszoros ütemezés

Többprocesszoros rendszer → a futásra kész folyamat több különböző processzoron is futhat

- *Heterogén rendszer:* van folyamat, amelyik csak bizonyos processzorokon futhat
- *Homogén rendszerek*: bármely folyamat, bármelyik processzoron futhat

Homogén többprocesszoros rendszerek

*Homogén rendszerek*ben a futásra kész folyamatokat közös sorokban tárolja.

- Szimmetrikus multiprocesszoros rendszer
  Minden CPU saját ütemezőt futtat, amely a közös sorokból választ. A sorok osztott használatához a kölcsönös kizárást biztosítani kell!
- Aszimmetrikus multiprocesszoros rendszer
  Az ütemező egy dedikált CPU-n fut, ez osztja szét a folyamatokat a szabad CPU-k között.

5.6 Az algoritmusok értékelésének módszerei

- Analitikus értékelés
  - Determinisztikus modellezés:
    - Előre elkészített adatokkal vizsgáljuk az algoritmus viselkedését.
    - Egyszerű, de az eredmények csak szűk körben érvényesek.
  - Sztochasztikus modellezés:
    - Az adateloszlások paraméterei adottak
    - Az algoritmusok matematikai eszközökkel vizsgálhatók
    - sorbanállás elmélete, queueing theory
- Szimuláció
  - Korábbi tapasztalatok alapján meghatározott véletlen eloszlású számokkal vizsgáljuk az algoritmus működését.
  - Az eredmények nagyban függnek az adatok helyességétől.
- Implementáció
  - A legmegbízhatóbb megoldás az algoritmusok implementálása és teljesítményük valós körülmények közötti mérése.
  - A legköltségesebb megoldás.

Windows ütemező

- Dinamikus prioritásos, preemptive multitasking
  - prioritás szintek
    - magasabb prioritású futásra kész, megszakítás
  - egy szinten belül időosztás
    - időszelet lejár, megszakítás
- 32 prioritási szint (0-31)
  - prioritás osztályok
- 0-s prioritás egyetlen (OS) thread

Windows ütemező

Relative Priority	Process Priority Class			
	Idle	Normal	High	Real Time
Idle	1	1	1	16
Lowest	2	6	11	22
Below Normal	3	7	12	23
Normal	4	8	13	24
Above Normal	5	9	14	25
Highest	6	10	15	26
Time Critical	15	15	15	31

Prioritás kiszámítása

- folyamat prioritása: 4, 8, 13, 24
- thread relatív prioritása: -2, -1, 0, +1, +2
- boost érték: 2, 1, 0

Priority level = base priority + boost

Unix ütemező (tradicionális)

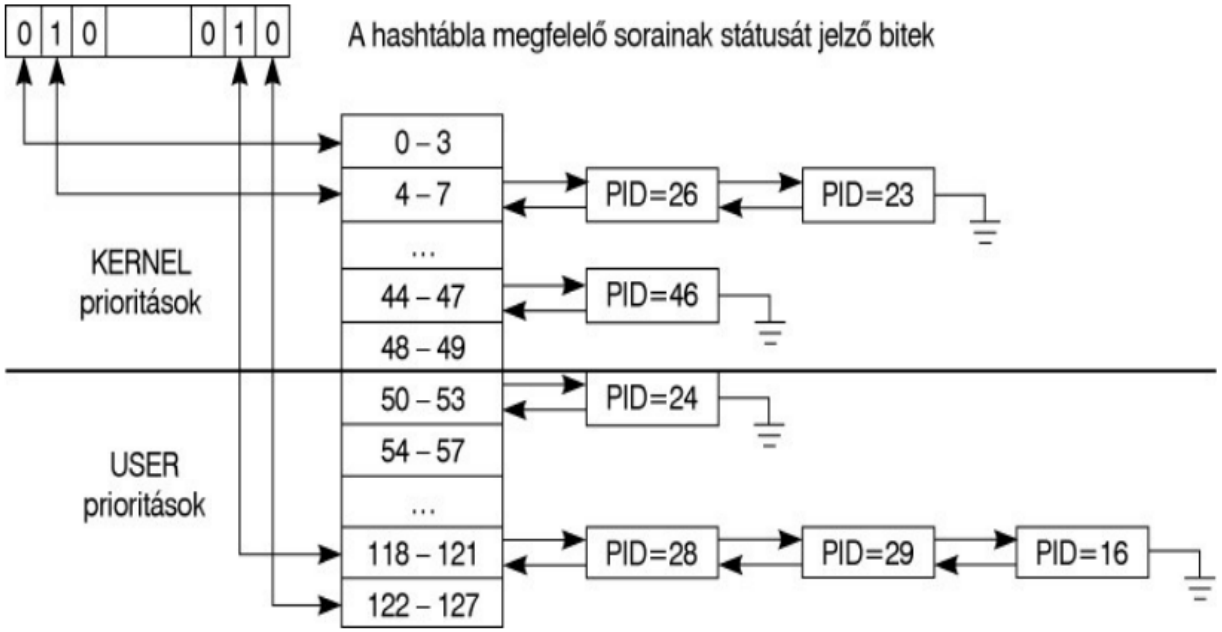
- User mode
  - nem preemptive, nincs időosztás, fix prioritás
- Kernel mode
  - preemptív, időosztásos, dinamikus prioritás
- Működés
  - prioritás szintek
    - magasabb prioritású futásra kész, megszakítás
  - egy szinten belül időosztás
    - időszelet lejár, megszakítás

Unix ütemező (tradicionális)

128 prioritási szint (0-127)

- 0-49: kernel
- 50-127: user

32 sor, soron belül időosztásos



Unix ütemező (tradicionális)

Prioritás számítás

- minden óraciklusban: p_pcu++
- minden 100. óraciklusban
  - p_cpu = p_cpu * KF
  - p_pri = P_USER + p_cpu / 4 + 2 * p_nice (P_USER = 50)

KF számítás (öregítés)

- SVR3
  - KF=1/2
- 4.3BSD
  - KF = 2 * load_avg / (2 * load_avg + 1)

Unix ütemező (tradicionális)

- Minden óraütésnél
  - Ha van magasabb prioritási sorban folyamat, akkor átütemezés
  - A futó folyamat p_cpu értékének növelése (csökkenni fog a prioritása)
- Minden 10. óraütésnél
  - Round-Robin ütemezés a legmagasabb (felhasználói) futási szinten levő folyamatok között
- Minden 100. óraütésnél
  - a korrekciós faktor kiszámítása az elmúlt 100 ciklus átlagos terhelése alapján
  - a p_cpu „öregítése”
  - a prioritás újraszámolása
  - szükség szerint átütemezés

Esettanulmány

- Completely fair scheduler
- Ingo Molnar
- http://immike.net/blog/2007/08/01/what-is-the-completely-fair-scheduler/